TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 6

Images & containers, Docker sidebar — Module 02 checkpoint + roster lab

Phase 1 — Foundations · Weeks 1–6 · Modules 00–02

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 02.04 Images vs Containers
2. 02.05 The VS Code Docker Sidebar
3. 02.06 Final Checkpoint
4. Live lab — Docker signup roster

Learning objective: Distinguish images from containers by running multiple containers from one image, then clean up both with `docker rm`, `docker rmi`, and the prune commands.

Lesson 04: Images vs Containers

By the end, you will be able to tell containers and images apart, and clean both up.

~20 minutes of typing

> **Image** = the recipe (read-only blueprint). **Container** = the meal cooked from it (a running or stopped instance). One image, many containers.

---

1. Start from home.

```
cd ~
```

2. See all your images.

```
docker images
```

You should see `hello-world` and `nginx`.

3. See all your containers (running and stopped).

```
docker ps -a
```

You should see your `hello-world` containers from lesson 02 and your `web` container from lesson 03.

4. Run nginx **three more times** with different names — each one is a separate container from the same image.

```
docker run -d -p 8081:80 --name web2 nginx
```

```
docker run -d -p 8082:80 --name web3 nginx
```

```
docker run -d -p 8083:80 --name web4 nginx
```

5. Look at running containers — three nginx containers, all from the **same image**.

```
docker ps
```

```
CONTAINER ID   IMAGE   ...   PORTS                   NAMES
...            nginx   ...   0.0.0.0:8083->80/tcp   web4
...            nginx   ...   0.0.0.0:8082->80/tcp   web2
...            nginx   ...   0.0.0.0:8081->80/tcp   web3
```

6. Confirm only one nginx **image** exists.

```
docker images | grep nginx
```

```
nginx   latest   ...   140MB
```

One image, multiple containers. This is the point.

7. Hit each one — they're independent processes.

```
curl -s http://localhost:8081 | grep title
curl -s http://localhost:8082 | grep title
curl -s http://localhost:8083 | grep title
```

Each prints `<title>Welcome to nginx!</title>`.

8. Stop them all at once.

```
docker stop web2 web3 web4
```

```
web2
web3
web4
```

9. Confirm they're stopped.

```
docker ps
```

(Empty — none running.)

10. They still exist on disk.

```
docker ps -a
```

You should see `web`, `web2`, `web3`, `web4` and the old `hello-world` containers — all with `Exited` status.

11. Remove the stopped nginx containers.

```
docker rm web web2 web3 web4
```

```
web
web2
web3
web4
```

12. Confirm they're gone.

```
docker ps -a | grep web
```

(No output — the web containers are gone.)

13. Remove the old hello-world containers in one command. `$(...)` runs the inner command first and pastes the output as arguments.

```
docker rm $(docker ps -aq --filter ancestor=hello-world)
```

You'll see the IDs of the removed containers printed.

14. Confirm only no containers remain.

```
docker ps -a
```

(Empty.)

15. The **images** are still there — removing containers doesn't remove the images they were created from.

```
docker images
```

You should still see `hello-world` and `nginx`.

16. Remove the hello-world image.

```
docker rmi hello-world
```

```
Untagged: hello-world:latest
Deleted: sha256:...
```

17. Try to run it again — it'll be re-pulled because it's gone.

```
docker run hello-world
```

You should see the "Unable to find image locally" line again, then the welcome text.

18. Clean up that container we just made too.

```
docker rm $(docker ps -aq)
```

19. A faster shortcut — `docker container prune` removes all stopped containers in one step.

```
docker container prune -f
```

```
Total reclaimed space: ...
```

20. Same idea for images: `docker image prune -a` removes images that aren't being used by any container.

```
docker image prune -a -f
```

This removes hello-world and nginx since nothing is using them. You'll see their layers being deleted.

21. Confirm everything is gone.

```
docker images
docker ps -a
```

Both should be empty (or nearly so).

22. Re-pull nginx — we'll need it for the final checkpoint.

```
docker pull nginx
```

23. Log it.

```
echo "Image vs container drills: complete" >> ~/docker-course/notes/notes.txt
```

Quick reference

Command	What it does
`docker images`	List images
`docker ps`	List **running** containers
`docker ps -a`	List **all** containers, running and stopped
`docker run <image>`	Create a new container from an image and start it
`docker stop <name>`	Stop a running container
`docker start <name>`	Start a stopped container
`docker rm <name>`	Remove a stopped container
`docker rmi <image>`	Remove an image
`docker logs <name>`	Show a container's logs
`docker exec -it <name> bash`	Get a shell inside a running container
`docker container prune -f`	Remove all stopped containers
`docker image prune -a -f`	Remove all unused images

Checkpoint

```
docker images | grep nginx
```

You should see nginx (you re-pulled it in step 22).

```
docker ps -a
```

Should be empty or near-empty — you cleaned up.

```
docker run -d -p 8080:80 --name test nginx && docker stop test && docker rm test
```

Should run, stop, and remove a container with no errors.

Lab

Objective: Tell images and containers apart — one image, many containers — and clean both up.

Task: Work steps 1–23 — run three more containers from the one nginx image, then remove containers with `docker rm`, images with `docker rmi`, prune the rest, re-pull nginx, and log it.

Submit: Paste the output of the three Checkpoint commands — `docker images | grep nginx`, `docker ps -a`, and `docker run -d -p 8080:80 --name test nginx && docker stop test && docker rm test` — plus `tail ~/docker-course/notes/notes.txt` into D2L.

Pass criteria:

- [ ] `docker images | grep nginx` shows the nginx image (re-pulled in step 22).
- [ ] `docker ps -a` is empty or near-empty — you cleaned up.
- [ ] The run/stop/remove one-liner completed with no errors.
- [ ] The notes tail includes `Image vs container drills: complete`.

Unlocks: 05-vscode-docker-sidebar.md

Next up: 05-vscode-docker-sidebar.md — See all this in VS Code's Docker sidebar

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Manage containers and images from the VS Code Docker sidebar, including stopping, removing, viewing logs, and attaching a shell via the right-click menu.

Lesson 05: The VS Code Docker Sidebar

Time: ~10 min

Remember Module 00 lesson 04? You installed the Docker extension in VS Code and the sidebar said "Failed to connect" because Docker wasn't installed yet. Now Docker is installed. Let's go look.

Step 1: open the Docker sidebar

1. In the activity bar (left edge of VS Code), click the **whale** icon (Docker).
2. The Docker side bar opens. The sections — **CONTAINERS**, **IMAGES**, **NETWORKS**, **VOLUMES**, **REGISTRIES** — should now be populated, not empty.

If they're still empty or showing an error, your VS Code session might be looking at the wrong Docker daemon (e.g. VS Code is on Windows but Docker is in WSL). Make sure your VS Code window is connected to the same environment where you installed Docker — bottom-left should say "WSL: Ubuntu" or "SSH: <hostname>." Reconnect if needed.

Step 2: run a container and watch it appear

1. Open the integrated terminal.
2. Run:

```
docker run -d -p 8080:80 --name sidebar-demo nginx
```

1. Look at the **CONTAINERS** section in the Docker sidebar.
2. `sidebar-demo` should appear in the list, with a green play icon (running).

You can refresh the sidebar manually if it doesn't auto-update — there's a refresh icon at the top of the Docker section.

Step 3: explore the right-click menu

1. Right-click `sidebar-demo` in the sidebar.
2. The context menu shows actions: **Stop**, **Restart**, **Remove**, **Inspect**, **Attach Shell**, **View Logs**, and more.
3. Click **View Logs** — VS Code opens nginx's logs in a new editor tab.
4. Click **Attach Shell** — VS Code opens a new integrated terminal that's already inside the container (same as `docker exec -it sidebar-demo bash`).

Type `exit` to leave the attached shell.

Step 4: stop and remove via the UI

1. Right-click `sidebar-demo` → **Stop**.
2. The icon next to the container changes from green to red.
3. Right-click → **Remove**.
4. The container disappears from the list.

You can do all of this from the CLI, and you should — it's faster once you know it. But the sidebar is a good way to **see** what's running at a glance, especially when you have multiple containers.

Step 5: explore the IMAGES section

1. Click the **IMAGES** section in the sidebar.
2. You should see `nginx` listed.
3. Right-click → see options like **Run**, **Run Interactive**, **Remove**.
4. **Run** is a quick way to start a container from an image without typing the full `docker run` command — though you lose control over flags like `-p` and `--name`. For real work, the CLI is better.

When to use the sidebar vs the CLI

Task	Better with
See what's running right now	Sidebar
Read a container's logs	Either — sidebar opens them in a tab, CLI is faster
Run a container with specific port mappings	CLI
One-off cleanup of stopped containers	CLI (`docker container prune -f`)
Get a shell into a running container	Either
Inspect environment variables / mounts	Sidebar (right-click → Inspect — much easier than `docker inspect`)

The sidebar is for seeing. The CLI is for doing. Use both.

Checkpoint

You're done when:

- [ ] You opened the Docker sidebar in VS Code and it showed populated sections (not "Failed to connect").
- [ ] You ran a container in the terminal and saw it appear in the sidebar.
- [ ] You stopped and removed a container via right-click in the sidebar.
- [ ] You opened a container's logs via the sidebar.

Lab

Objective: Drive containers from VS Code's Docker sidebar, and know when the sidebar beats the CLI (and when it doesn't).

Task: Work Steps 1–5 — open the Docker sidebar, run `sidebar-demo` from the terminal and watch it appear, view its logs and attach a shell from the right-click menu, then stop and remove it via the UI.

Submit: One screenshot of the populated Docker sidebar showing the **CONTAINERS** and **IMAGES** sections — take it while `sidebar-demo` is running so it appears under CONTAINERS, with `nginx` visible under IMAGES.

Pass criteria:

- [ ] Sidebar shows populated CONTAINERS and IMAGES sections (not "Failed to connect").
- [ ] `sidebar-demo` is visible under CONTAINERS with the green running icon.
- [ ] You stopped and removed the container via right-click and opened its logs from the sidebar.

Unlocks: 06-final-checkpoint.md

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Demonstrate module mastery by running, verifying, and cleaning up a named detached nginx container end-to-end without instructor help.

Lesson 06: Final Checkpoint

Time: ~15 min — **no instructor help.**

Self-test. Re-read earlier lessons if you forget something. Do not ask the instructor or your neighbor.

The test

Type these commands. Each step assumes the prior step succeeded.

1. Confirm Docker is installed and you're in the docker group.

```
docker --version && groups | grep -q docker && echo "OK: docker group" || echo
"FAIL: not in docker group"
```

Expected: a Docker version + `OK: docker group`.

2. Make sure no leftover containers are running.

```
docker ps -aq | xargs -r docker rm -f
```

3. Run nginx in detached mode on port 8080, named `final`.

```
docker run -d -p 8080:80 --name final nginx
```

4. Confirm it's running.

```
docker ps --filter name=final
```

Expected: one row with status `Up` and the port `0.0.0.0:8080->80/tcp`.

5. Hit it with curl.

```
curl -s http://localhost:8080 | grep -o "Welcome to nginx"
```

Expected: `Welcome to nginx`.

6. Look at the logs — they should show your curl request.

```
docker logs final | tail -n 3
```

Expected: at least one line mentioning `GET /`.

7. Open VS Code's Docker sidebar (whale icon in the activity bar). Confirm `final` is listed under **CONTAINERS** with a green/running indicator.

8. Take a screenshot showing, in one image:

- VS Code's Docker sidebar with the `final` container visible and running.
- Your integrated terminal with the output of `docker ps --filter name=final`.

9. Clean up.

```
docker stop final && docker rm final
```

10. Log it.

```
echo "Module 02 final checkpoint: passed $(date -Iseconds)" >> ~/docker-course/notes/notes.txt
```

Lab

This is the Module 02 phase-gate lab — passing it completes Phase 1.

Objective: Prove, unaided, that you can run, verify, inspect, and clean up a Docker container end to end.

Task: Work the ten test steps above in order, alone — no instructor, no neighbor. Re-read earlier lessons if you forget a command.

Submit: The one screenshot from step 8: VS Code's Docker sidebar showing the `final` container running **and** your integrated terminal showing the output of `docker ps --filter name=final`.

Pass criteria:

- [ ] All commands ran without errors (no `permission denied`, no `command not found`).
- [ ] Step 5 returned `Welcome to nginx`.
- [ ] Your screenshot clearly shows `final` running in VS Code's Docker sidebar.
- [ ] The same screenshot shows `docker ps --filter name=final` in the terminal with status `Up`.

If any of those failed, redo the relevant lesson and try again.

Unlocks: Phase 2 — Module 03 FastAPI (phase gate)

What's next

Module 03 is **FastAPI** — Phase 2 begins, and you'll build and containerize a real API. Docker stays installed — you'll keep using it across the rest of the course. Most later modules run things in containers.

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Live lab — Docker signup roster

Wednesday live lab · 6:00–8:00 PM · three-person teams · does **not** replace the Module 02 / nginx phase-gate (02:06)

> **Learning objective:** Run a two-service Compose stack (API + Postgres) so both containers are healthy, the host browser shows the roster, and `psql` matches the UI — without ever starting Python on the host.

Everything runs in Docker: the API (`web`) and Postgres (`db`). Do not run `app.py` or uvicorn on the host. This is the stretch Compose lab from cohort 1 — leftover nginx from the Module 02 gate will steal port 8080, so clean that up first.

Get the lab

Either clone the public student repo, or unzip the attached packet.

```
git clone https://github.com/timcarrender04/docker-roster-lab.git
cd docker-roster-lab
```

Packet (same files, no git): docker-roster-lab.zip. Unzip, then File → Open Folder on that directory.

Before you type

- VS Code connected to the same Linux as Docker (bottom-left: `WSL: Ubuntu` or `SSH: …`). The whale sidebar must not say Failed to connect.
- You can run Docker without sudo (`docker run hello-world` already worked).
- Three-person teams. Name a leader. Everyone clones and runs Compose on their own machine — no watching-only.
- Suggested split: (1) Docker — compose, logs, sidebar · (2) Research — docs/Hub + `RESEARCH.txt` · (3) Validate — curl, browser, `compose exec psql`.
- Grok or another LLM *outside* the IDE is allowed. Still cite a real doc/Hub URL in `RESEARCH.txt`.

Clock

Clock	What happens
6:00	Sign-in, form rooms of 3, name a leader, assign roles.
6:10	Clone, Open Folder, sidebar, free 8080, read compose.
6:25	Fill `RESEARCH.txt` (docs/Hub; Grok in a browser is OK).
6:45	`docker compose up -d --build`. Wait for db healthy.
7:15	Browser, curl API, `compose exec psql`, sidebar screenshot.
7:45	Leaders: 3/3 up? One stuck spot.
7:55	D2L: screenshot + `RESEARCH.txt` (three names).

Free port 8080, then bring it up

```
docker ps
docker ps -aq | xargs -r docker rm -f

# wrong: python3 app.py    /    uvicorn app:app ...
# right:
docker compose up -d --build
docker compose ps
```

Open `docker-compose.yml` and `Dockerfile`. Two services: `web` builds this folder and serves the form + API on 8080; `db` is official `postgres:16`. `app.py` is copied *into* the web image. It is not a host command.

Prove both containers

Browser: http://localhost:8080 — add a name + email. The table should show the row.

```
curl -s http://localhost:8080 | grep -i signup
curl -s -X POST http://localhost:8080/api/signups \
  -H "Content-Type: application/json" \
  -d '{"name":"Ada","email":"ada@example.com"}'
docker compose exec db psql -U roster -d roster -c "SELECT name, email FROM
signups;"
docker compose logs web --tail=20
```

Whale sidebar should show `web` and `db` running.

What “done” looks like

- `docker compose ps` shows web Up and db healthy (or Up).
- GET/POST `/api/signups` hit the web container, not a local Python process.
- From web, Postgres hostname is the service name `db` (not localhost).
- Named volume `pgdata` keeps rows after `docker compose down` then `up` (`down -v` wipes data).
- You did not run python/uvicorn on the host.
- Three names on the team; each person has `compose ps` green.

Fill every question in `RESEARCH.txt` from *your* compose file and the linked docs before you treat the lab as done. Official references:

- Compose file: docs.docker.com/compose/compose-file
- Compose CLI: docs.docker.com/compose
- Postgres image: hub.docker.com/_/postgres
- Dockerfile: docs.docker.com/reference/dockerfile
- Compose networking: docs.docker.com/compose/how-tos/networking

Submit. One screenshot: VS Code Docker sidebar showing both services running, terminal `docker compose ps`, and the browser roster with at least one row. Also submit `RESEARCH.txt` (your answers, not a blank copy) with all three names.